18-740: Modern Computer Architecture & Design

Lecture 4:
"Wide Instruction Fetch and Dynamic Decode"

John Paul Shen

September 3, 2026

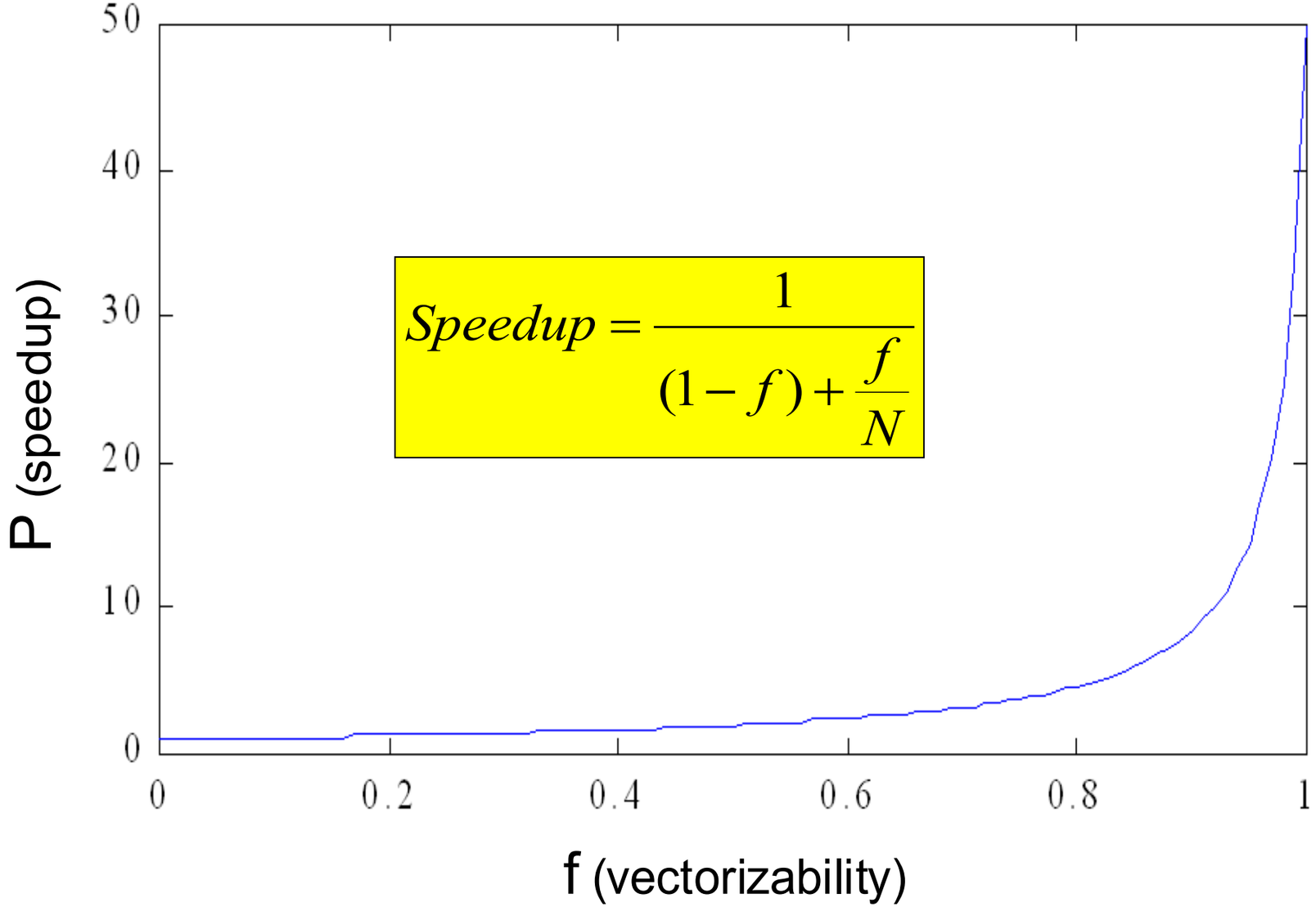
- Required Reading Assignments:
  - **Chapter 4 and Chapter 5 of Shen and Lipasti (SnL)**
- Recommended Reference:
  - Eric Rotenberg, S. Bennett, and James E. Smith. *"Trace Cache: A Low Latency Approach to High Bandwidth Instruction Fetching."* MICRO, 1996.

Electrical & Computer ENGINEERING



"Tyranny of Amdahl's Law"

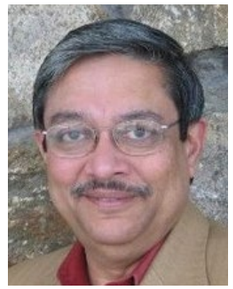
[Bob Colwell, CMU-Intel-DARPA]



- Suppose that a computation has a 4% serial portion, what is the limit of speedup on 16 processors?
  - 1/((0.04) + (0.96/16)) = 10
  - What is the maximum speedup?
    - 1/0.04 = 25 (with N → ∞)

[Tilak Agerwala and John Cocke, 1987]

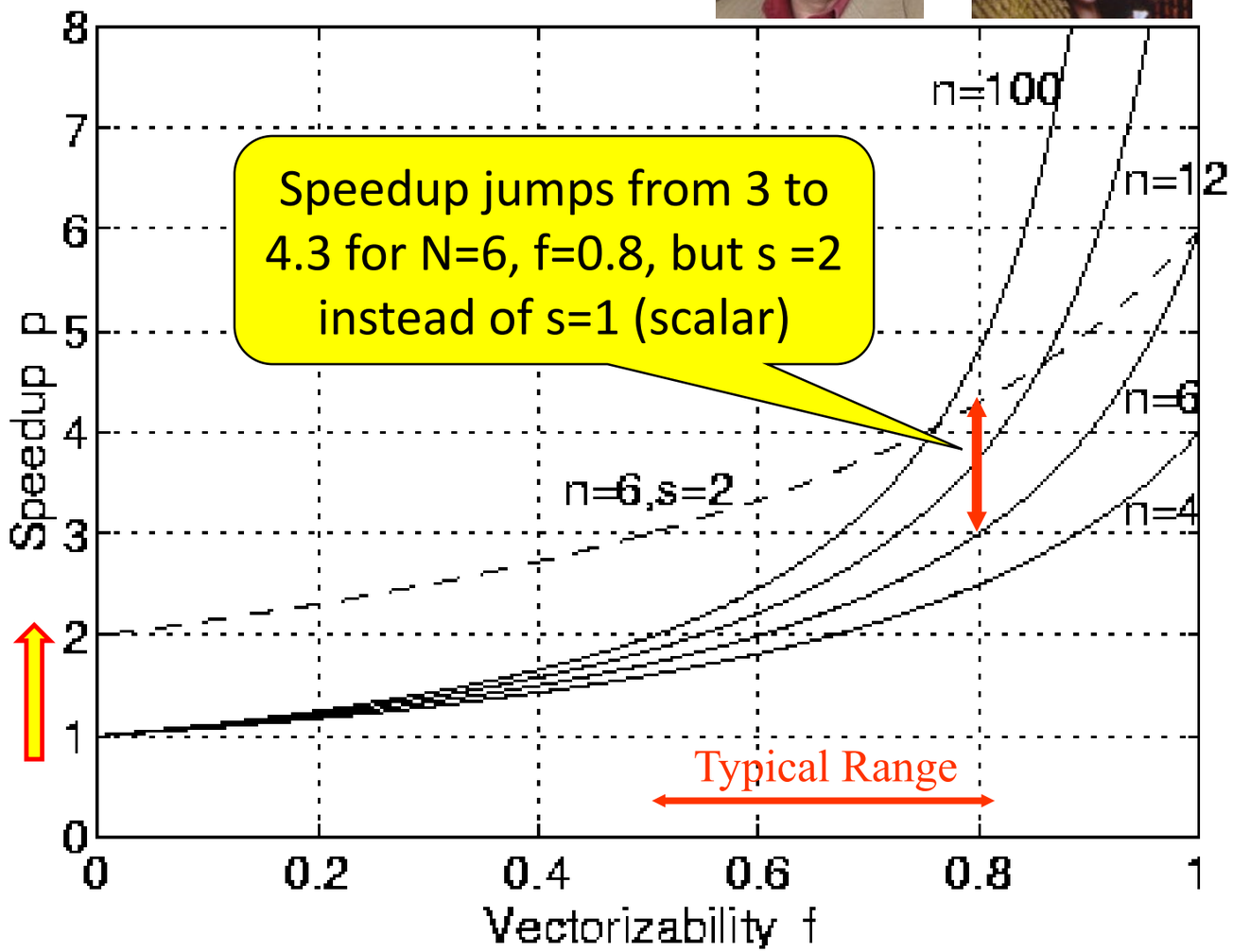
Motivation for Superscalar (ILP) Design





- **Moderate the tyranny of Amdahl's Law**
  - Ease the sequential bottleneck
  - More generally applicable
  - Robust (less sensitive to *f*)
  - Revised Amdahl's Law:

$$Speedup = \frac{1}{\frac{(1-f)}{S} + \frac{f}{N}}$$



Limits on Instruction-Level Parallelism (ILP)

Weiss and Smith [1984]	1.58
Sohi and Vajapeyam [1987]	1.81
Tjaden and Flynn [1970]	1.86 (“Flynn’s bottleneck”)
Tjaden and Flynn [1973]	1.96
Uht [1986]	2.00
Smith et al. [1989]	2.00
Jouppi and Wall [1988]	2.40
Johnson [1991]	2.50
Acosta et al. [1986]	2.79
Wedig [1982]	3.00
Butler et al. [1991]	5.8
Melvin and Patt [1991]	6
Wall [1991]	7 (Jouppi disagreed)
Kuck et al. [1972]	8
Riseman and Foster [1972]	51 (no control dependencies)
Nicolau and Fisher [1984]	90 (“Fisher’s optimism”)



"Iron Law" of Processor Performance

$$1/\text{ComputerPerformance} = \frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Time}}{\text{Cycle}}$$

(inst. count) (CPI) (cycle time)

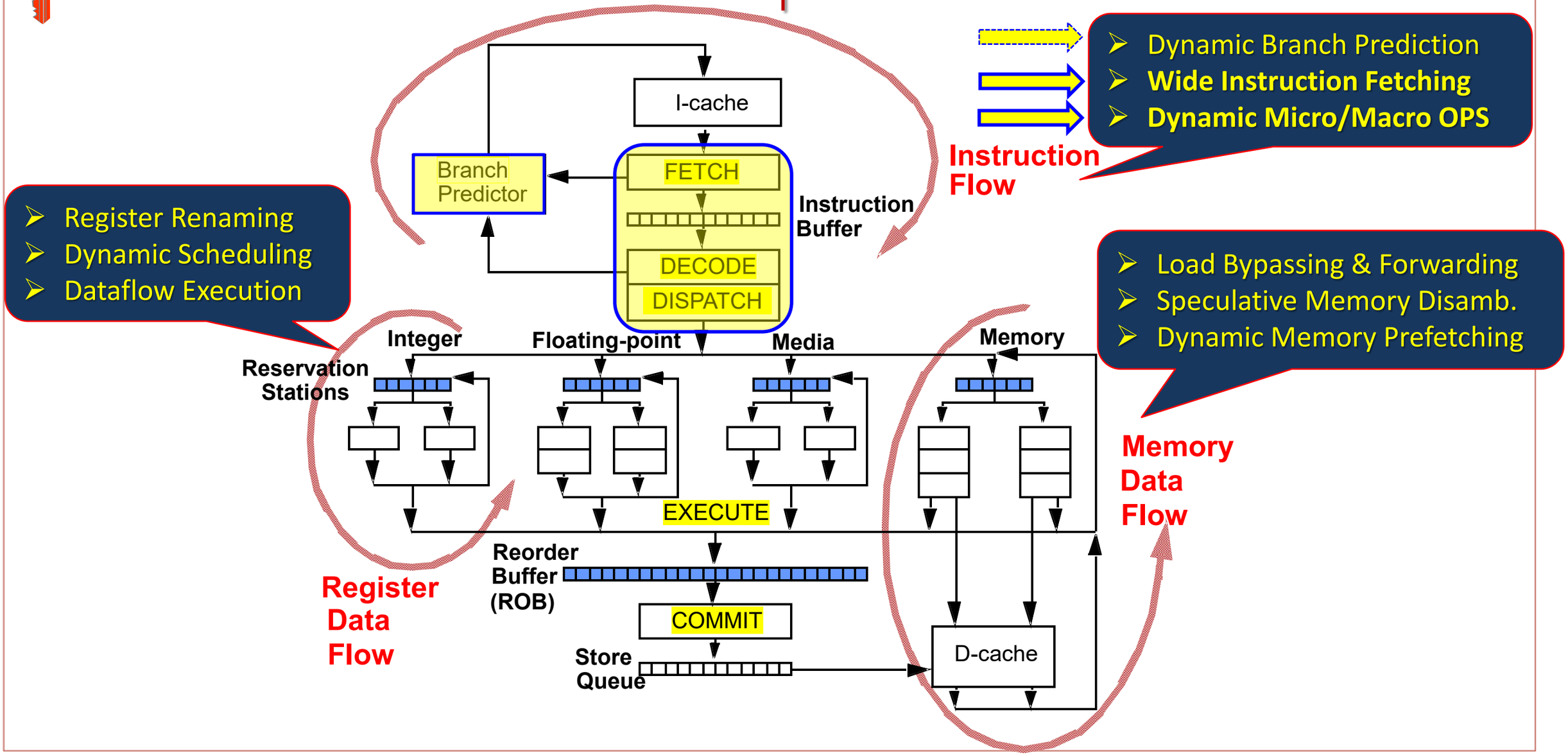
Architecture → Implementation → Realization

Compiler Designer Processor Designer Chip Designer

- In the 1980's: **(pipelining)**
  - CPI: 5.0 → 1.2
- In the 1990's: **(superscalar)**
  - CPI: 1.2 → 0.5
  - **IPC: 0.8 → 2.5**
- In the 2000's: **(multicores)**
  - Hit power wall !!!
  - ILP → TLP

[Mike Johnson, *Superscalar Microprocessor Design*, Prentice-Hall, 1991]

Three Flow Paths of Superscalar Processors



- Dynamic Branch Prediction
- **Wide Instruction Fetching**
- **Dynamic Micro/Macro OPS**

- Register Renaming
- Dynamic Scheduling
- Dataflow Execution

- Load Bypassing & Forwarding
- Speculative Memory Disamb.
- Dynamic Memory Prefetching

18-740: Modern Computer Architecture & Design

Lecture 4:
"Wide Instruction Fetch and Dynamic Decode"

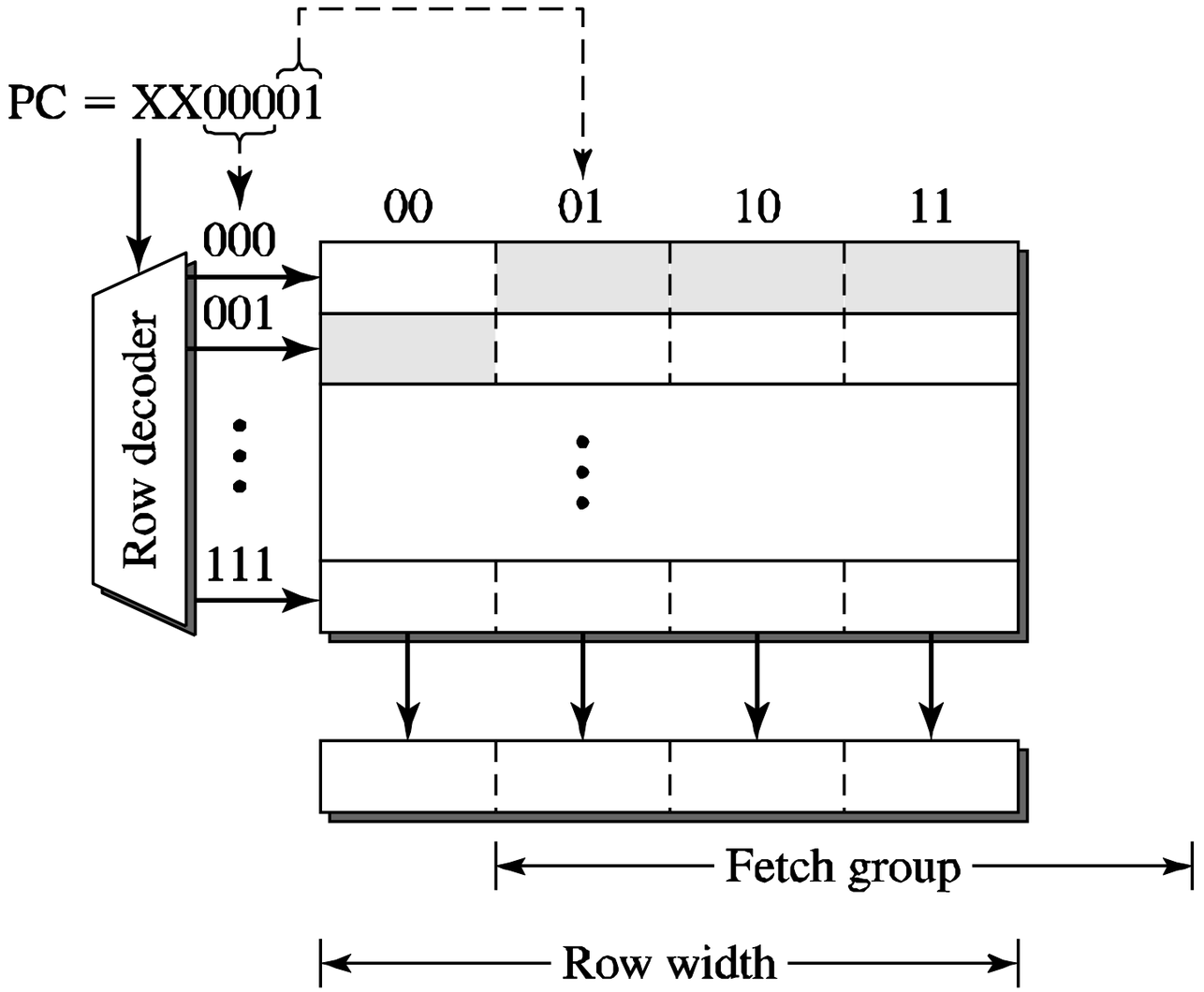
A. **Wide Instruction Fetching**
   a. Fetch Alignment Problem
   b. Trace Cache & Loop Buffer
B. Dynamic Instruction Decoding

Electrical & Computer ENGINEERING

Wide Instruction Fetching

- For a N-way superscalar, need ≥ N-way fetching
  - Otherwise, N-way ILP can never be achieved
  - Sometimes, wider than N-way fetch helps – why?
- Implementation: wide port to I-cache
  - Read many words from I-cache
  - Select those from current PC to first taken branch
- Impediments to wide instruction fetching
  - Instruction cache misses
  - Instruction group alignment
  - Complex instruction set (e.g. x86)
  - Branches and jumps

The Fetch Alignment Problem



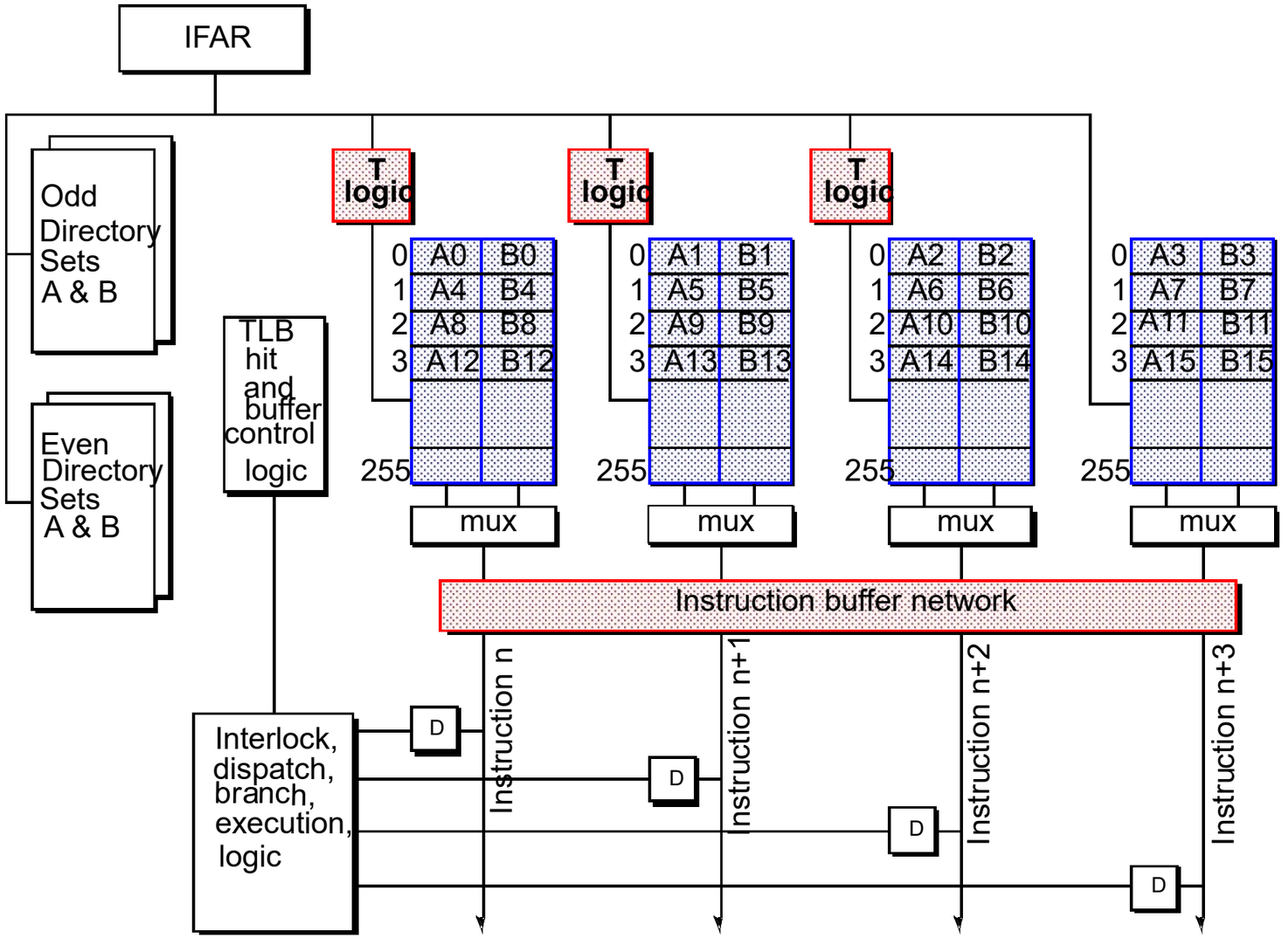
Solving the Alignment Problem

- Software solution: align taken branch targets to I-cache row starts
  - Effect on code size?
  - Effect on the I-cache miss rate?
  - What happens when we go to the next chip?

- Hardware solution
  - Detect (mis)alignment case
  - Allow access of multiple rows (current and sequential next)
    - True multi-ported cache, over-clocked cache, multi-banked cache, …
    - Or keep around the cache line from previous access
      - Assuming a large basic block
  - Collapse the two fetched rows into one instruction group

IBM RS/6000 I-Cache and Fetch (auto alignment)

- 2-way set associative (A and B sets) I-Cache; (8) 256-instruction SRAM modules
- 16 instruction per cache line (64 bytes)

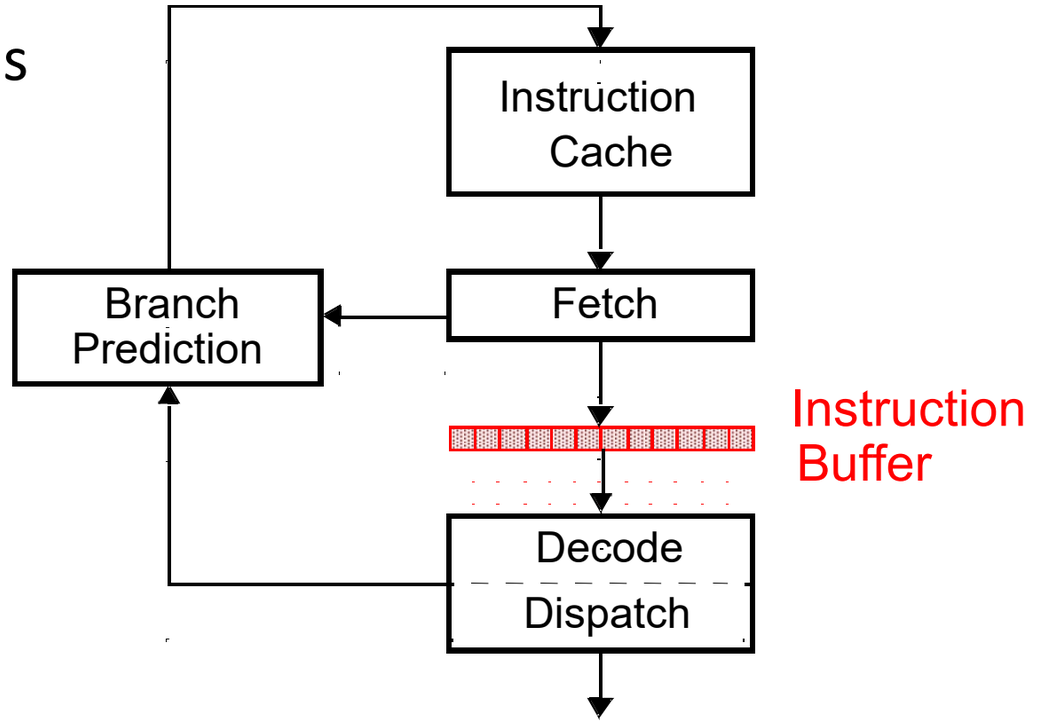


Cache Line Underutilization

- What if we have <N instructions between two taken branches?
  - Or predicted taken branches?
- Solution: read multiple cache lines
  - Current and predicted next cache line
  - Merge instructions from two cache lines using a collapsing buffer
  - Question: how do we get the predicted next cache line address?
    - Need two predictions (PCs) per cycle
- Easier cases to handle
  - Intra-cache line forward & backward branches

Impediments to Wide Fetching

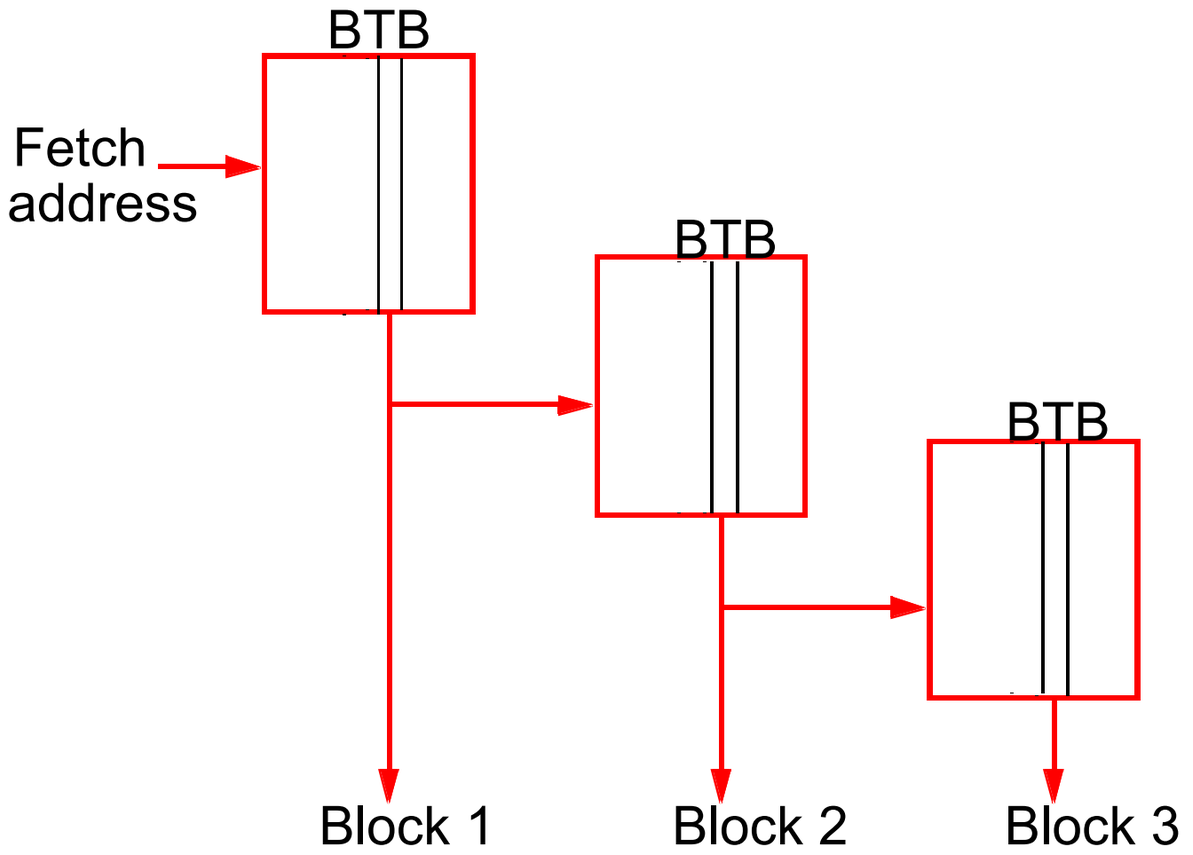
- Average Basic Block Size
  - integer code: 4-6 instructions
  - floating-point code: 6-10 instructions
- Three Major Challenges:
  - Multiple-Branch Prediction
  - Multiple Fetch Groups
  - Alignment and Collapsing



Cannot be solved with just longer cache blocks

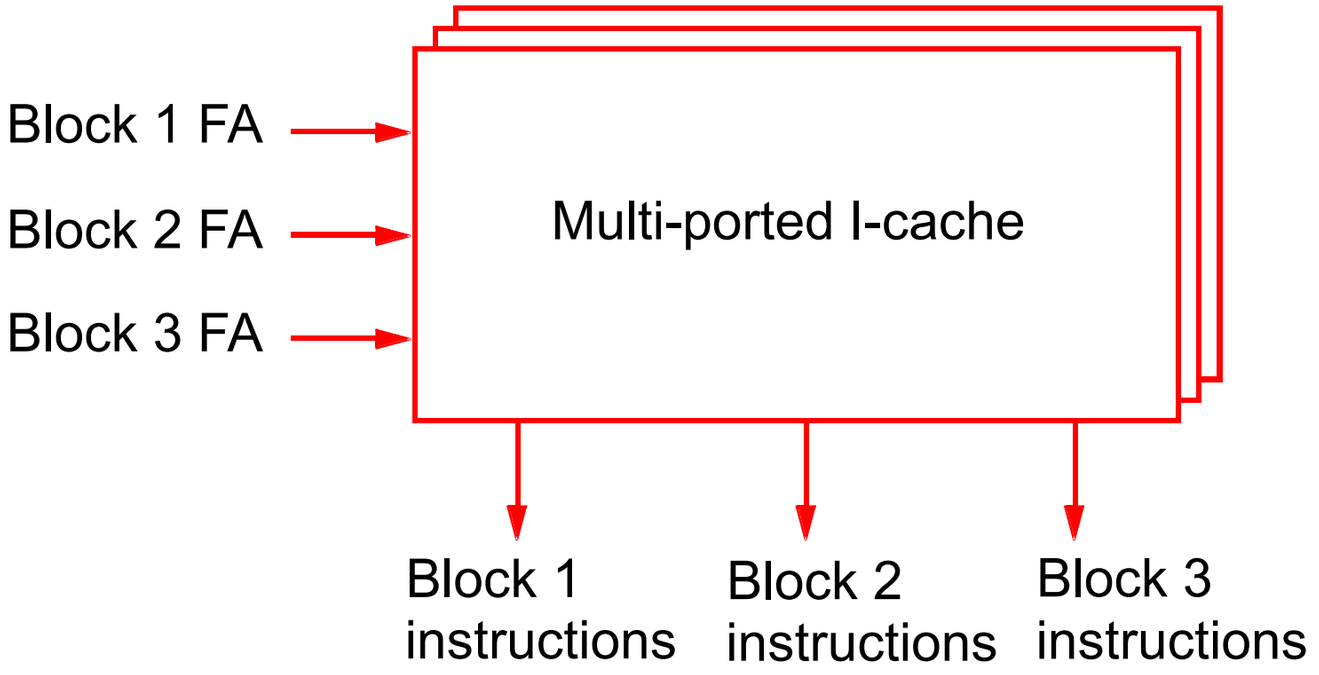
Multiple Branch Predictions

- Issues with multiple branch predictions:
  - Latency resulting from sequential predictions
  - Later predictions based on stale/speculative history
  - Don't forget, 0.95x0.95x0.95=0.85



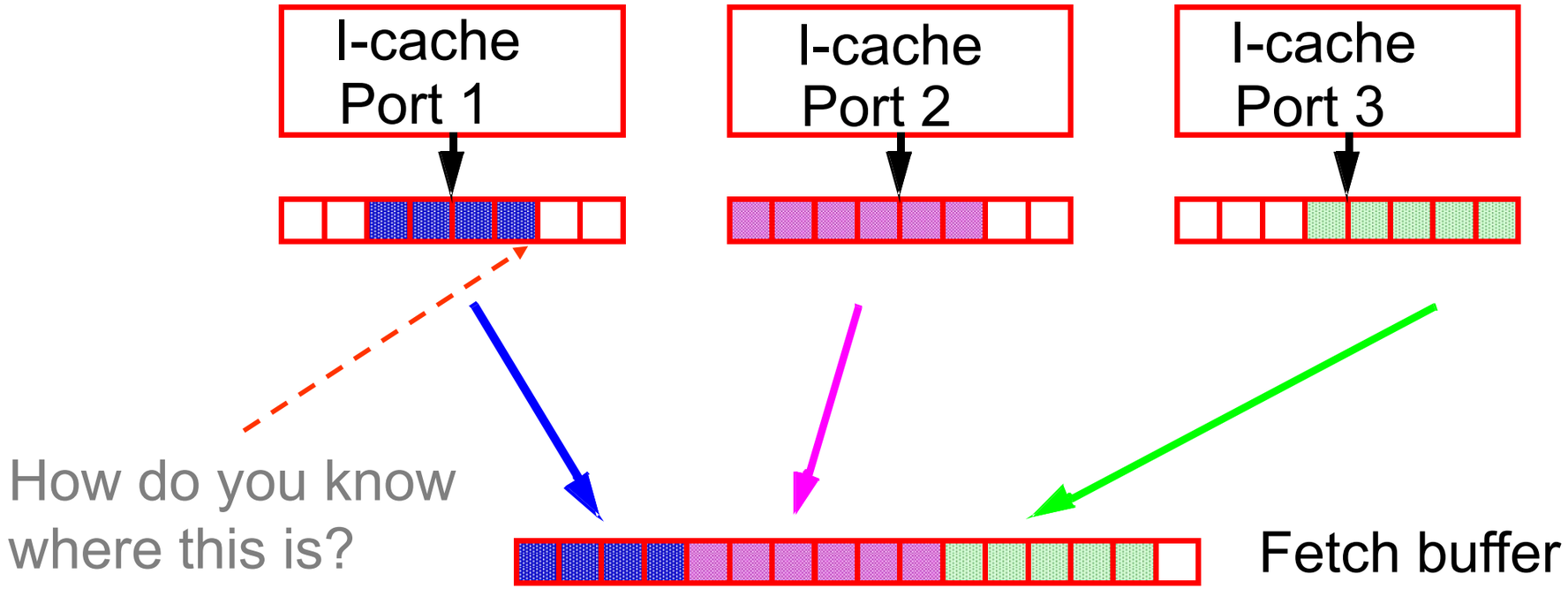
Multiple Predicted Taken Branches

- Issues with multiple taken branches:
  - Long latency with multiple sequential I-cache accesses
  - or, multi-ported I-cache with slower access latency
  - or, multi-banked I-cache to approximate multi-port



Instruction Alignment and Collapsing

- Issues with alignment and collapsing:
  - Misalignment between fetch group and cache line.
  - Packing of variable-sized blocks into fetch buffer.

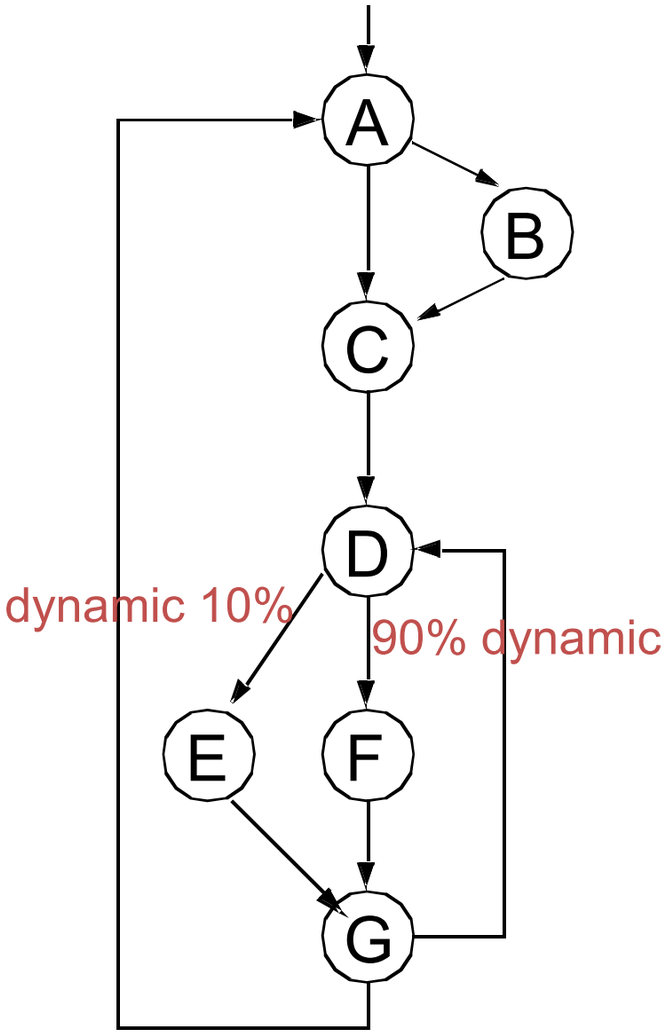


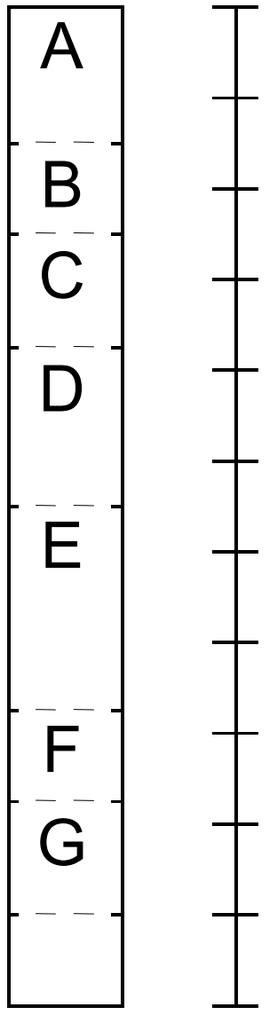
The Trace Cache Proposal

Eric Rotenberg, S. Bennett, and James E. Smith. *"Trace Cache: A Low Latency Approach to High Bandwidth Instruction Fetching."* MICRO, 1996.

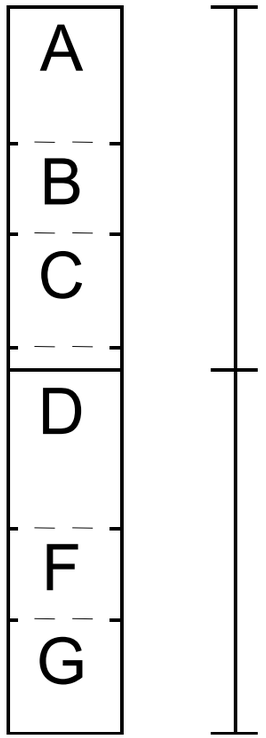


Eric Rotenberg



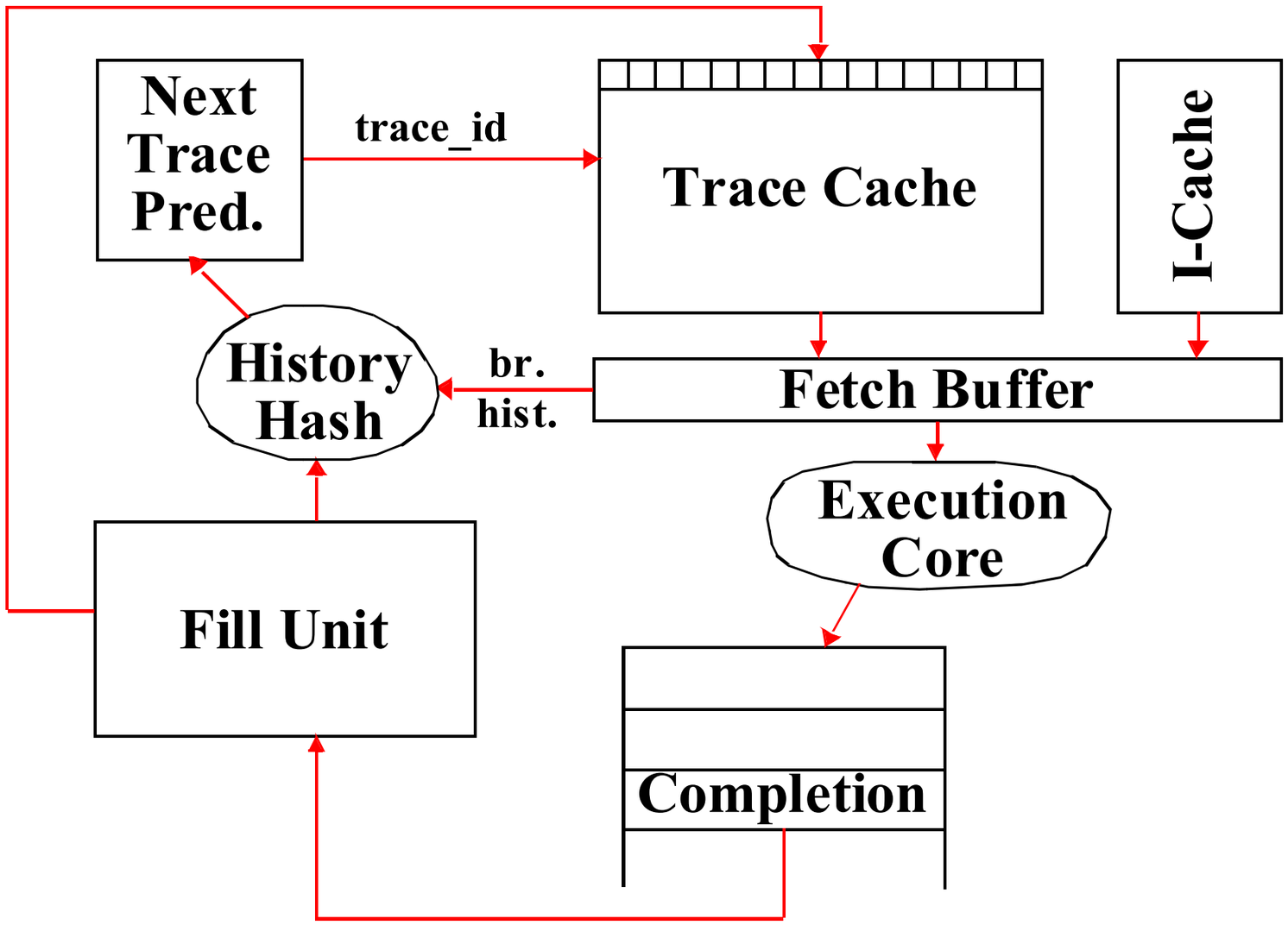


I-cache line boundaries



Trace-cache line boundaries

A Typical Trace Cache Organization



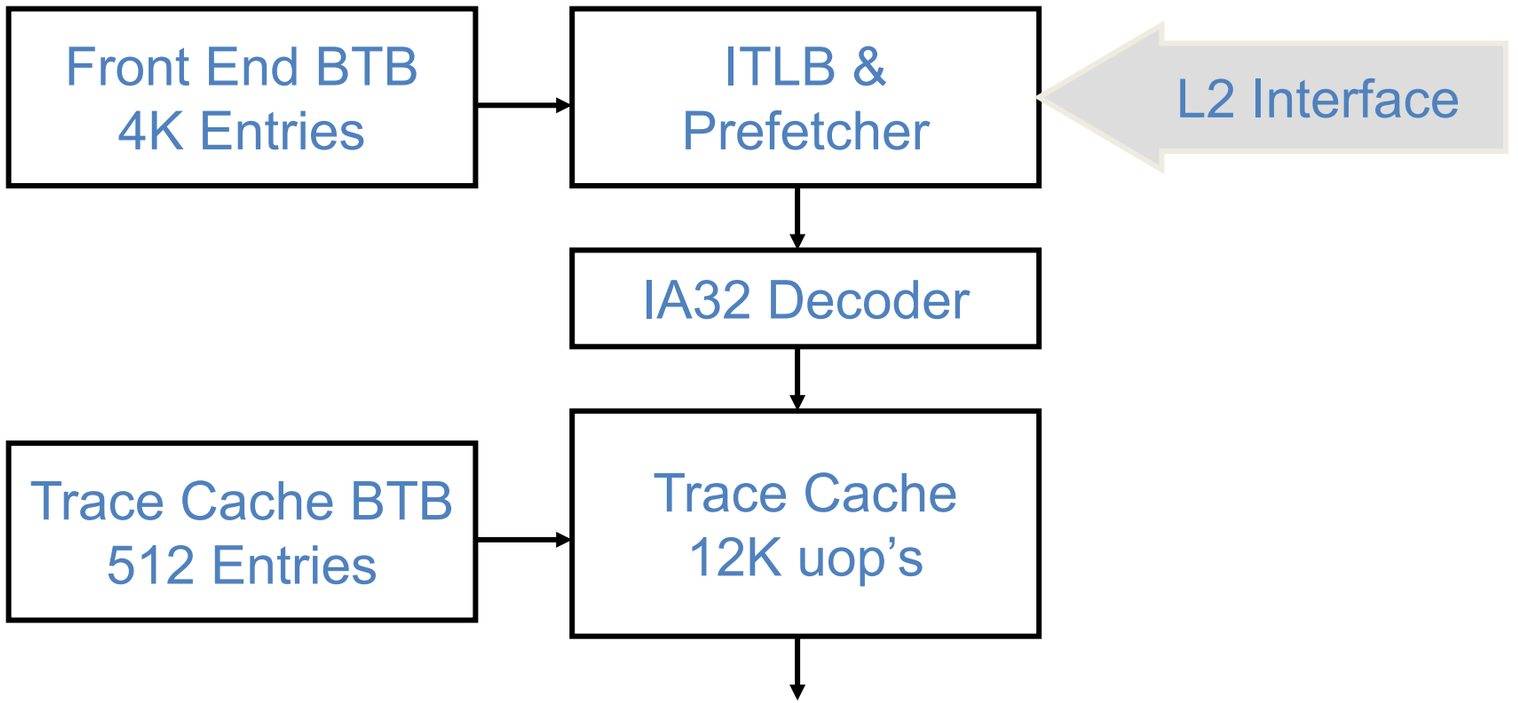
Trace Fill Unit

- Observe the dynamic execution sequence
- Gather instructions into a trace segment (or trace cache block)
- Some simple heuristics for forming trace segments
  - stop after collecting up to *N* instructions (*N* is the trace cache block size)
  - stop after *B* conditional branches (*B* is the limit of the multi-branch predictor)
  - stop after seeing an register-indirect jump
  - Don’t split basic blocks
  - In some designs, unconditional and conditional branches can be dropped from the traces
- Can include pre-decoded dependency information
- Can even dynamically re-order instructions.

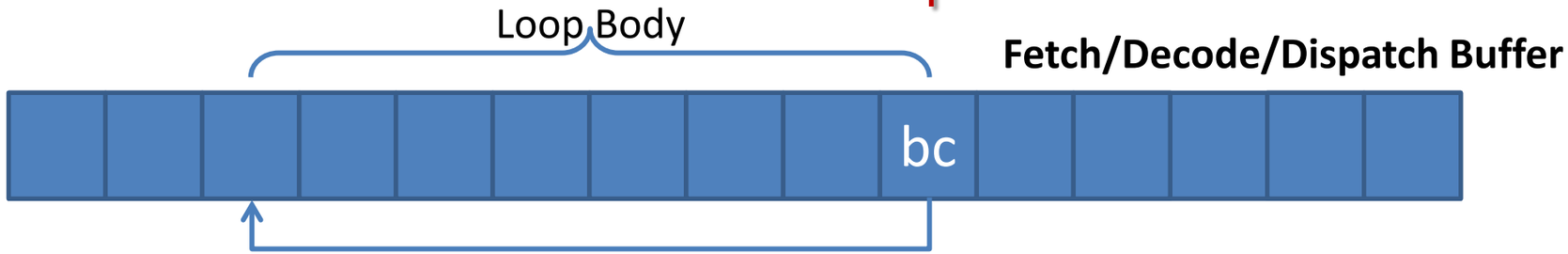
Intel Pentium 4 Trace Cache

- A 12K-uop trace cache replaces the L1 I-cache
- 6-uop per trace line, can include branches
- Trace cache returns 3-uop per cycle
- IA-32 decoder can be simpler and slower

Only needs to decode one IA-32 instruction per cycle



High-Bandwidth Fetch: Loop Buffers



- History: AMD29K Branch Target Cache
  - Don't cache the target address; cache 4 instructions from the target itself
  - Avoid accessing I$ for first fetch group following a taken branch
  - If loop body is <= 4 instructions, effectively a loop cache
  - Room for 32/64 branch targets
- Introduced in Intel Merom (Core 2 Duo)
  - Fetch buffer detects short backward branches, inhibits refetch from I$
- Intel Nehalem (Core i7)
  - Moved loop buffer after decode: contains uops
- Intel Sandybridge
  - General-purpose uop cache (not just loops)
  - 1.5K capacity

18-740: Modern Computer Architecture & Design

Lecture 4:
"Wide Instruction Fetch and Dynamic Decode"

A. Wide Instruction Fetching
   a. Fetch Alignment Problem
   b. Trace Cache & Loop Buffer

B. Dynamic Instruction Decoding

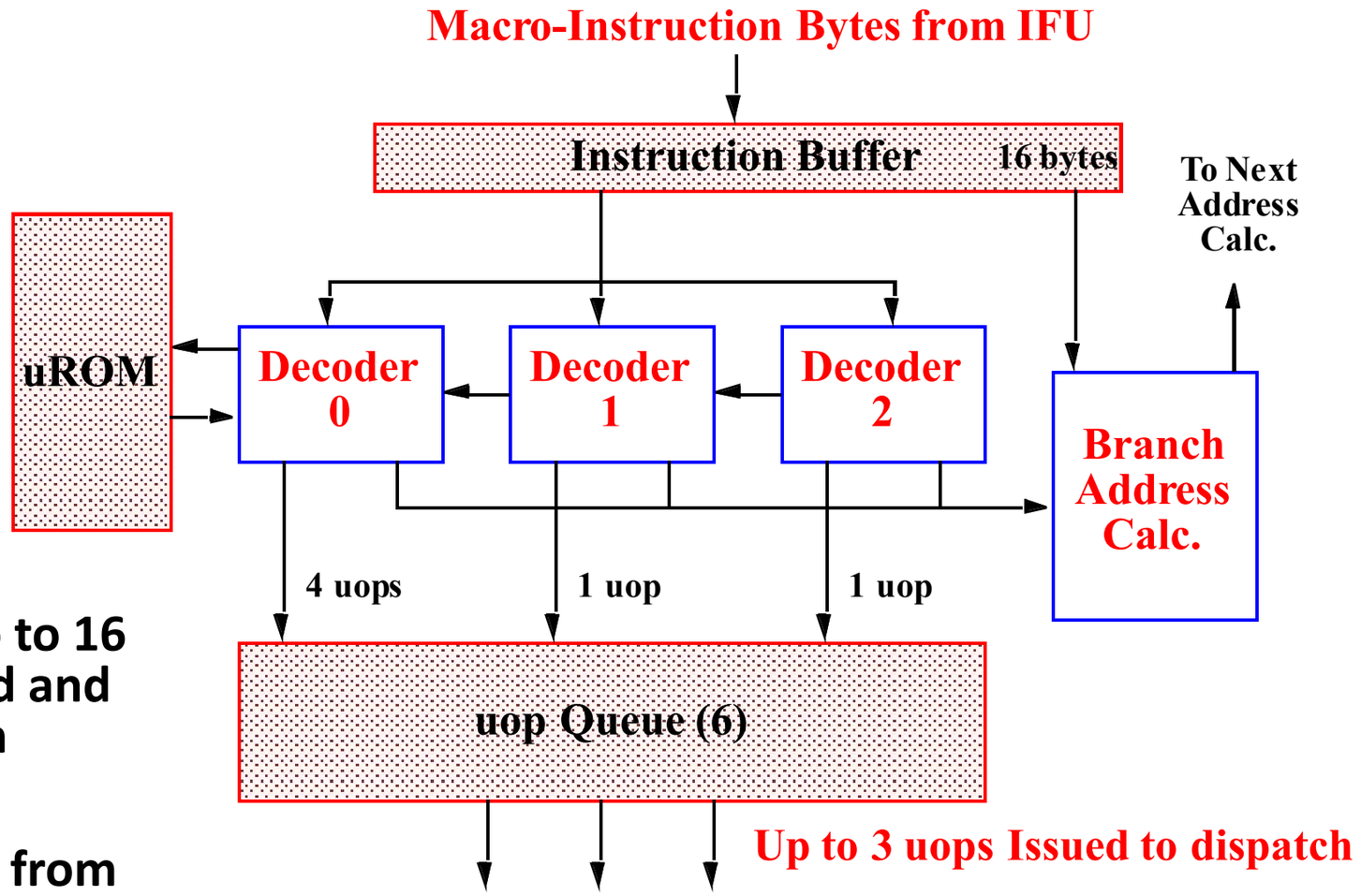
Electrical & Computer ENGINEERING

Instruction Decoding Issues

- Primary decoding tasks
  - Identify individual instructions for CISC ISAs
  - Determine instruction types (especially branches)
    - Drop instructions after (predicted) taken branches
    - Potentially restart the fetch pipeline from target address
  - Determine dependencies between instructions
- Two important factors
  - Instruction set architecture (RISC vs. CISC)
    - Determines decoding difficulty
  - Pipeline width
    - Sets the number of comparators for dependency detection

Intel Pentium Pro (P6) Instruction Decode

- Branch instruction detection
- Branch address calculation - Static prediction and branch always execution
- One branch decode per cycle (break on branch)



- Instruction Buffer contains up to 16 bytes, which must be decoded and queued before the instruction buffer is re-filled
- Macro-instructions must shift from decoder 2 to decoder 1 to decoder 0

What is a uop?

Small two-operand instruction - Very RISC like.

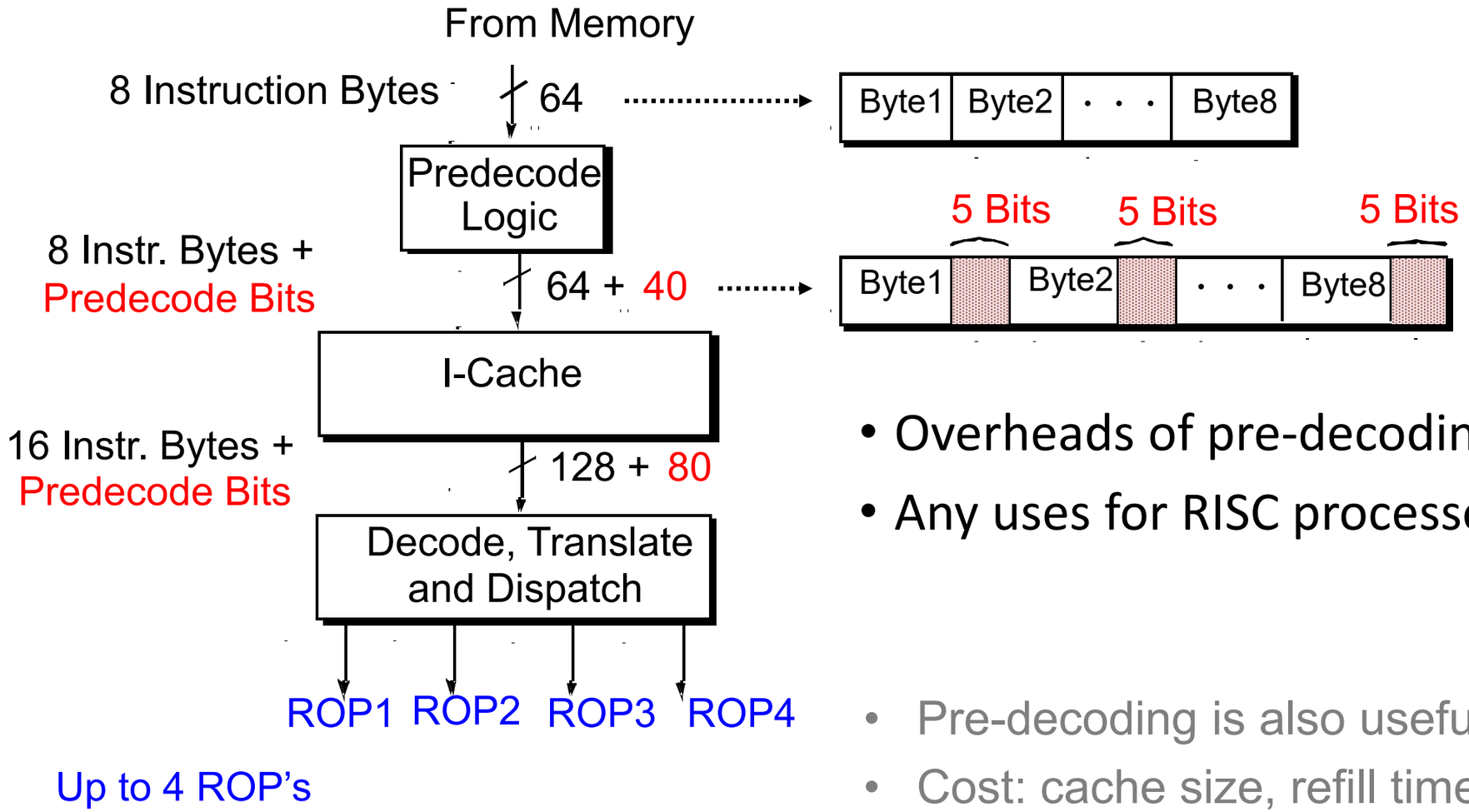
IA-32 instruction

add (eax),(ebx)	MEM(eax) <- MEM(eax) + MEM(ebx)

Uop decomposition:

ld guop0,(eax)	guop0 <- MEM(eax)
ld guop1,(ebx)	guop1 <- MEM(ebx)
add guop0,guop1	guop0 <- guop0 + guop1
sta eax	
std guop0	MEM(eax) <- guop0

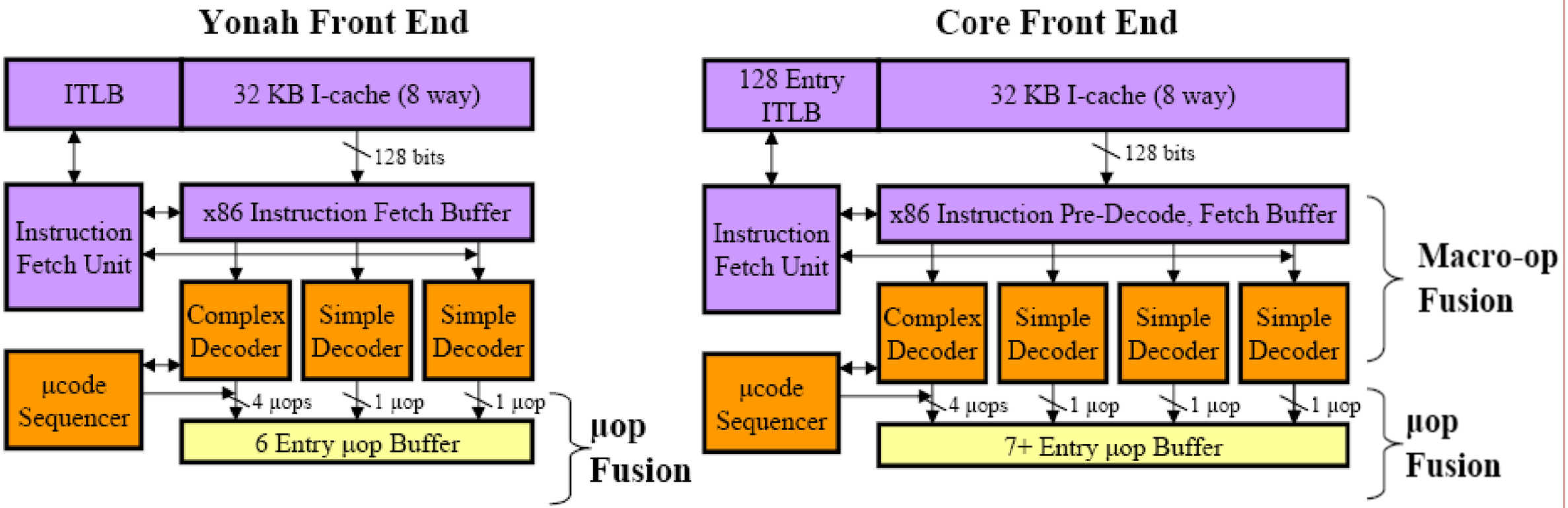
I-Cache Pre-decoding in the AMD K5



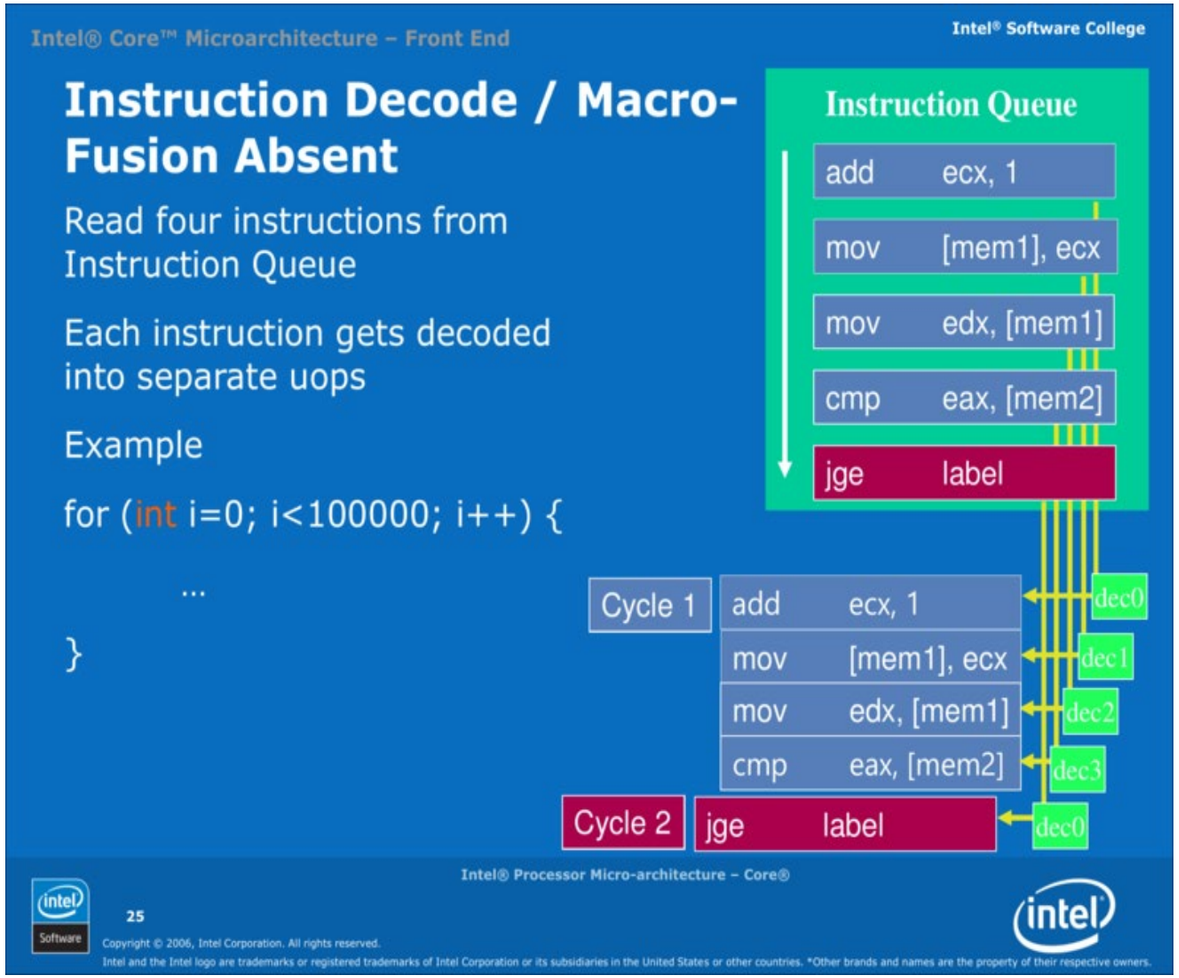
- Overheads of pre-decoding?
- Any uses for RISC processors?

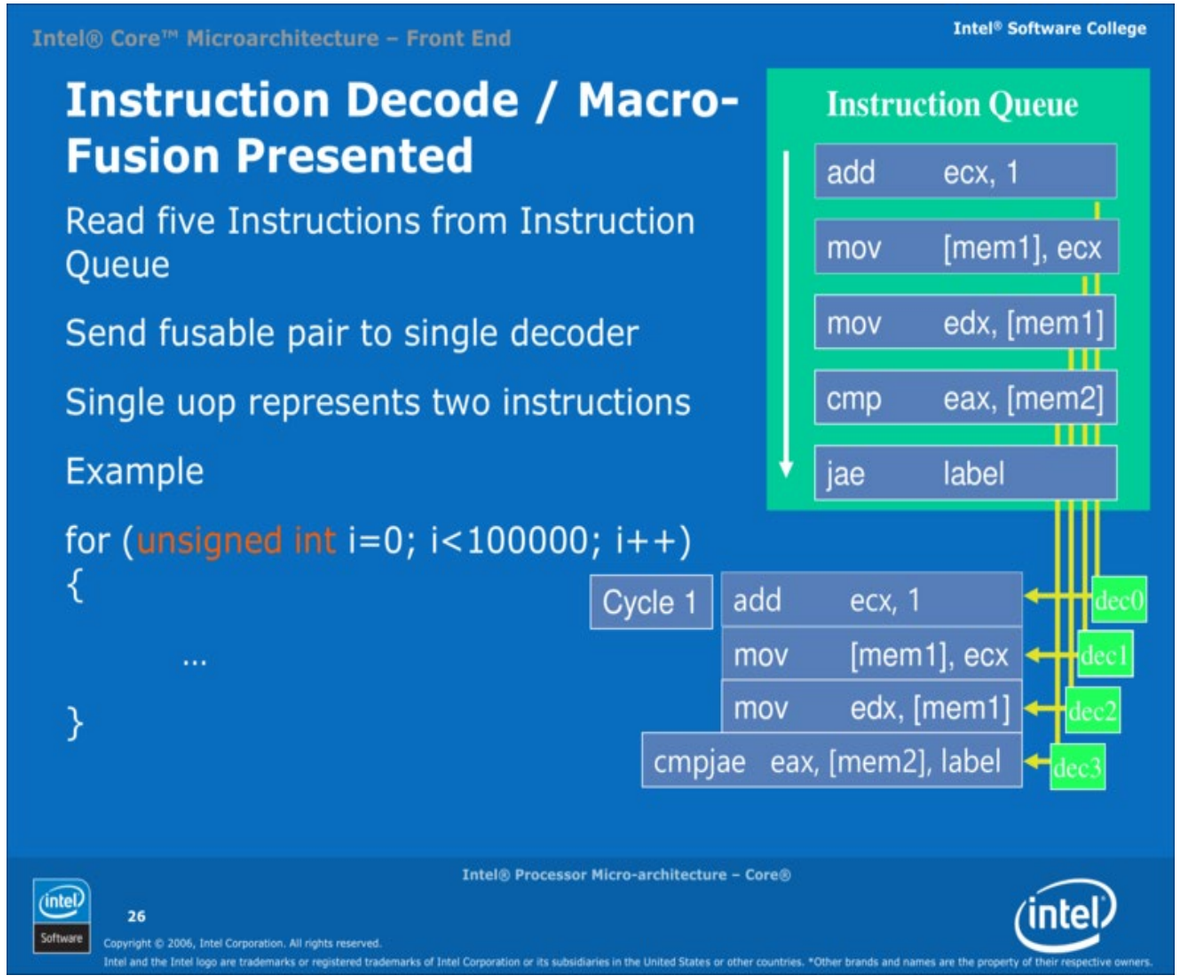
- Pre-decoding is also useful for RISC
- Cost: cache size, refill time

Micro-Op Fusion and Macro-Op Fusion



Macro-Op Fusion Example on Intel x86





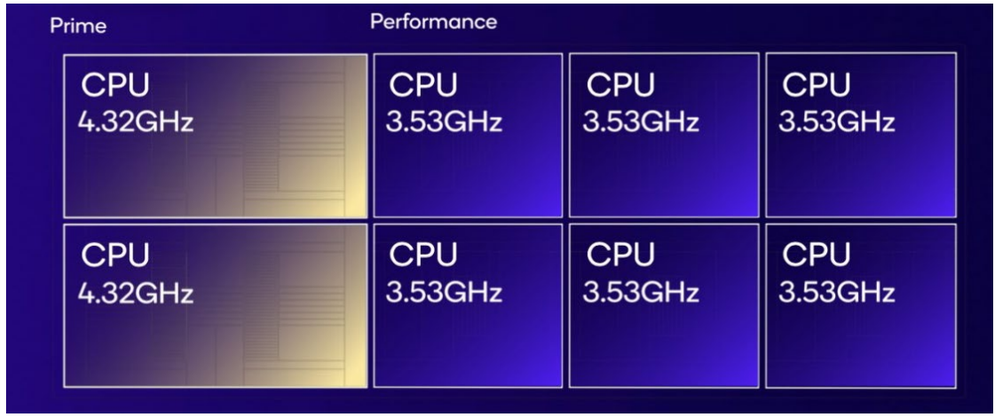
https://www.youtube.com/watch?v=hA_IrCYyyE4

Recent Qualcomm Snapdragon Mobile SoCs



https://www.msn.com/en-us/technology/hardware-and-devices/why-mobile-gamers-are-calling-this-the-best-snapdragon-yet/vi-AA1RHZ8Z?ocid=socialshare

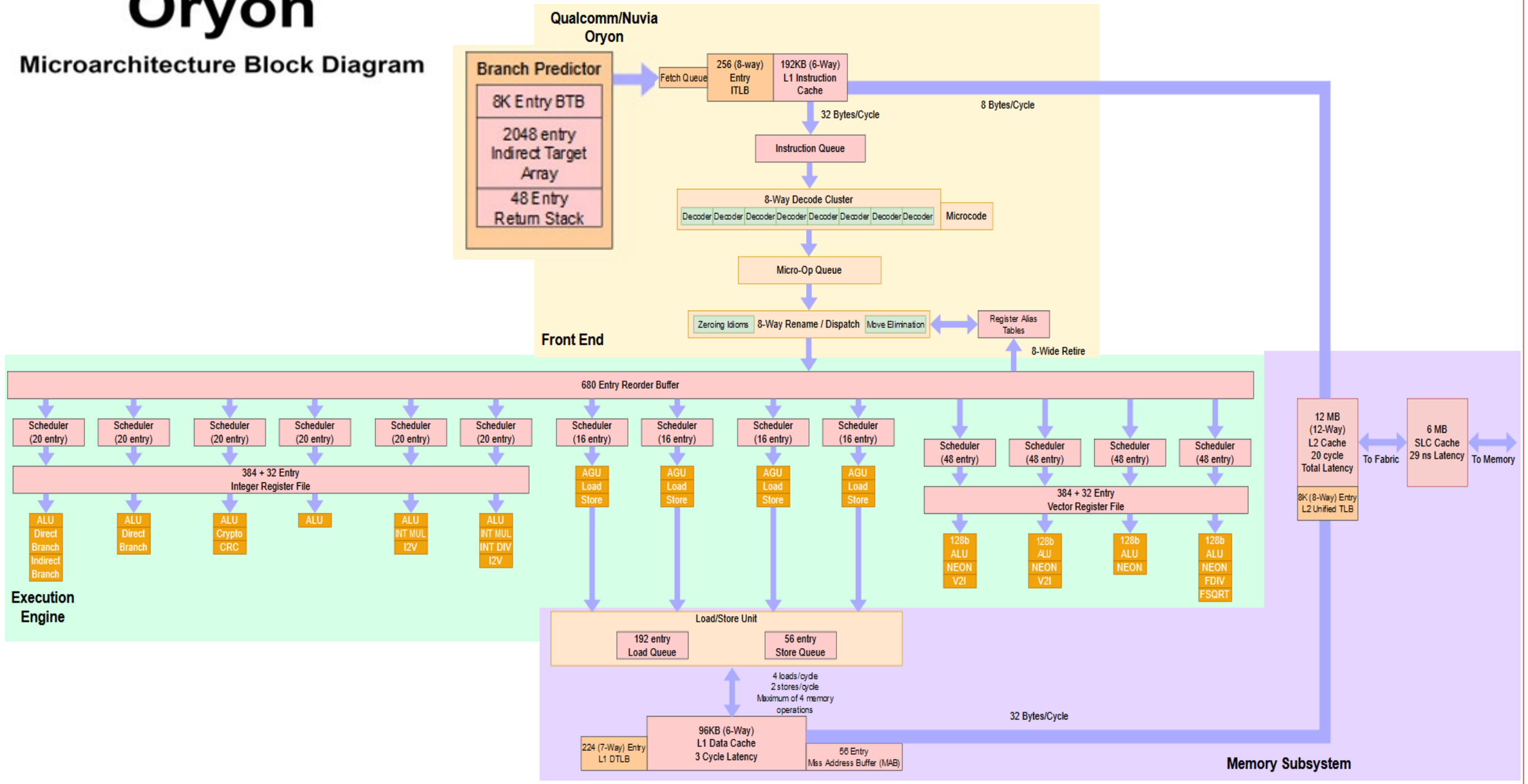




	Snapdragon 8 Elite	Snapdragon 8 Gen 3	Snapdragon 8 Gen 2
CPU Cluster Configuration	Oryon CPU Cores: 2x 4.32GHz (Phoenix-L) 6x 3.53GHz (Phoenix-M)	1x 3.3GHz (Cortex-X4) 3x 3.2GHz (Cortex-A720) 2x 3GHz (Cortex-A720) 2x 2.3GHz (Cortex-A520 Refresh)	1x 3.19GHz (Cortex-X3) 2x 2.8GHz (Cortex-A715) 2x 2.8GHz (Cortex-A710) 3x 2.0GHz (Cortex-A510)
GPU	Adreno (ray tracing support)	Adreno (ray tracing support)	Adreno 740 (ray tracing support)
NPU	Hexagon (fused scalar, tensor, and vector) Mixed precision INT8/INT16 INT4 support	Hexagon (fused scalar, tensor, and vector) Mixed precision INT8/INT16 INT4 support	Hexagon (fused scalar, tensor, and vector) Mixed precision INT8/INT16 INT4 support
RAM support	LPDDR5X	LPDDR5X	LPDDR5X
Process	TSMC 3nm	TSMC 4nm (N4P)	TSMC 4nm (N4)

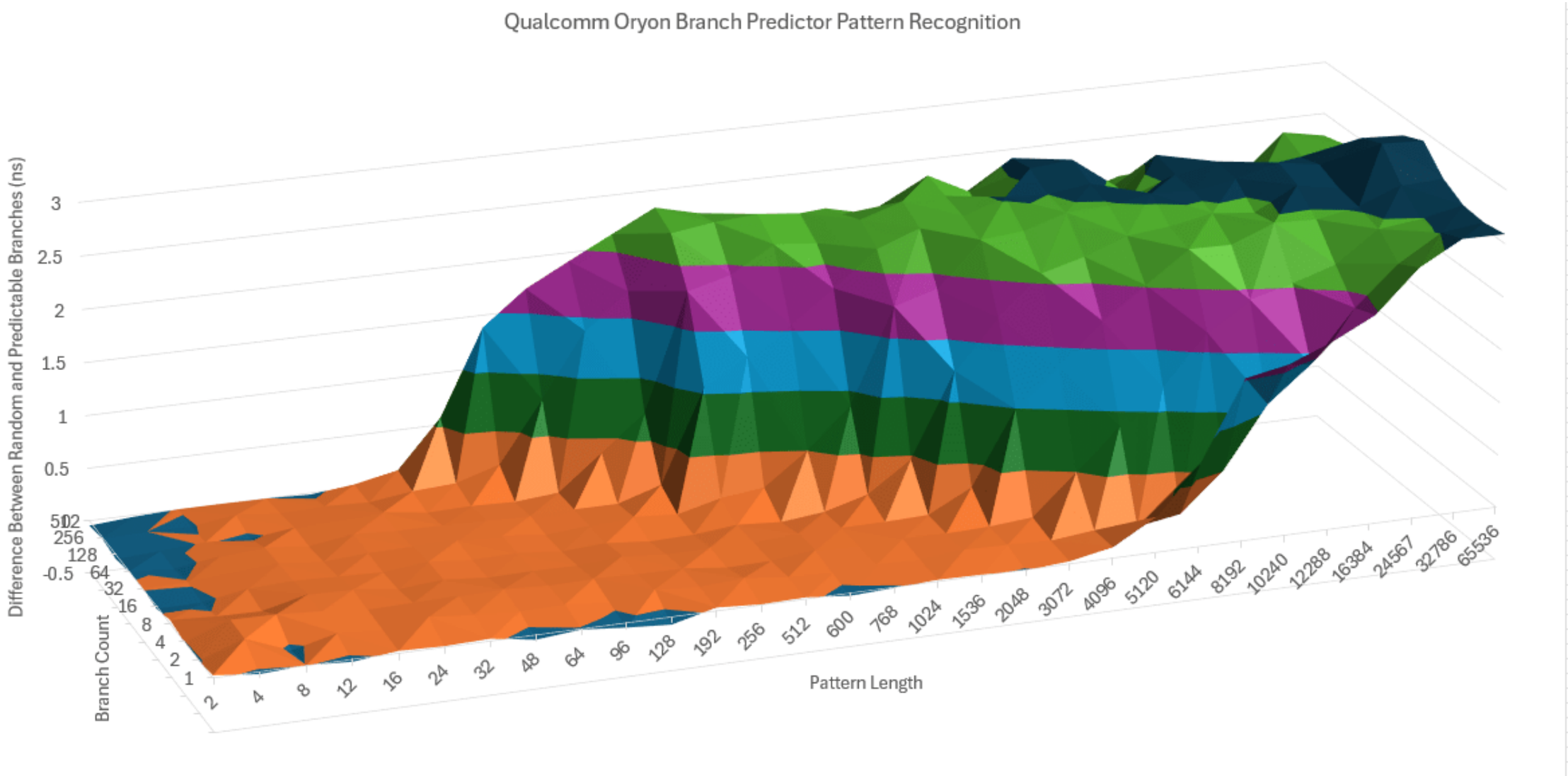
Oryon

Microarchitecture Block Diagram



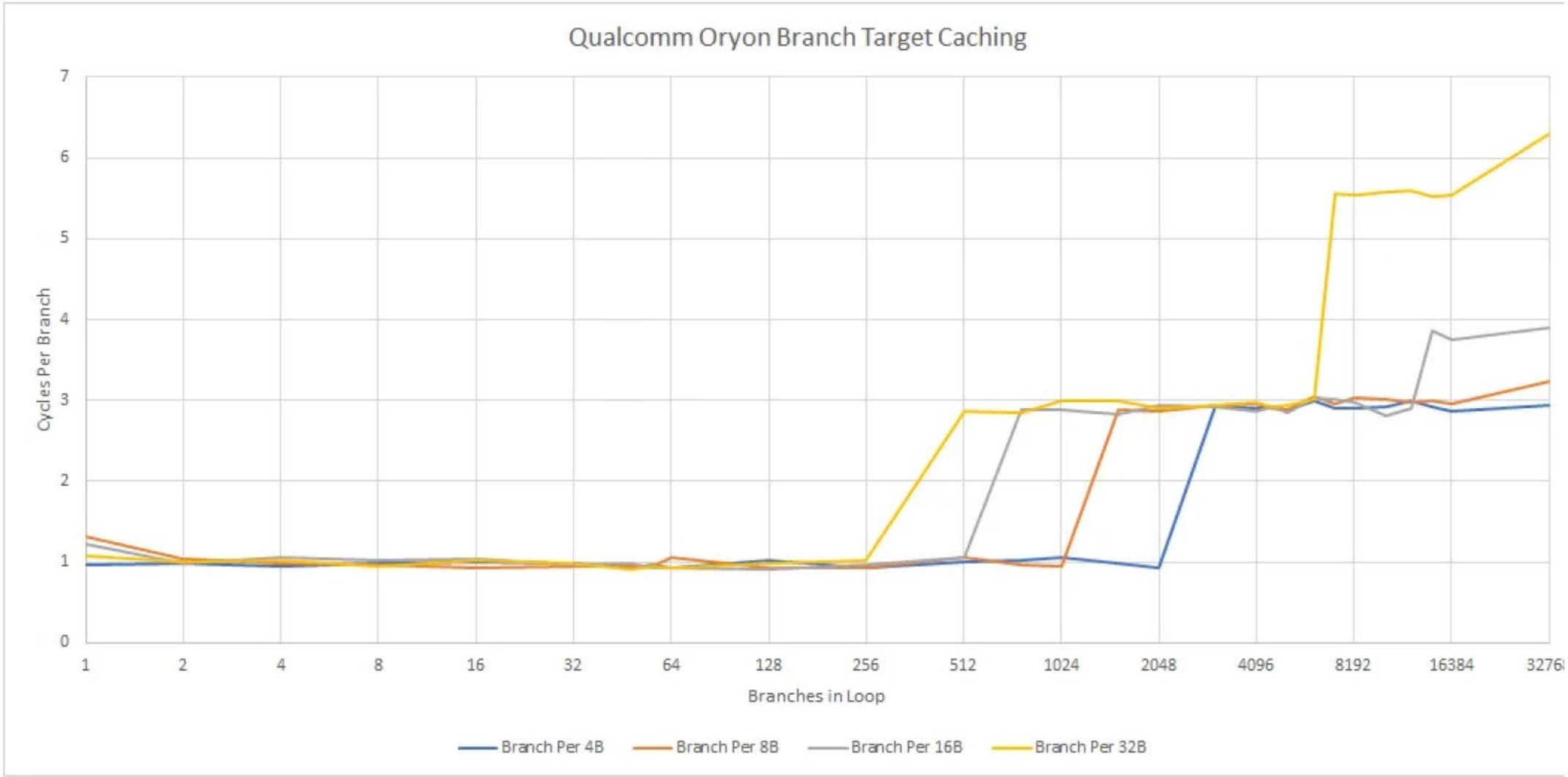
https://chipsandcheese.com/p/qualcomms-oryon-core-a-long-time-in-the-making

Oryon's direction predictor



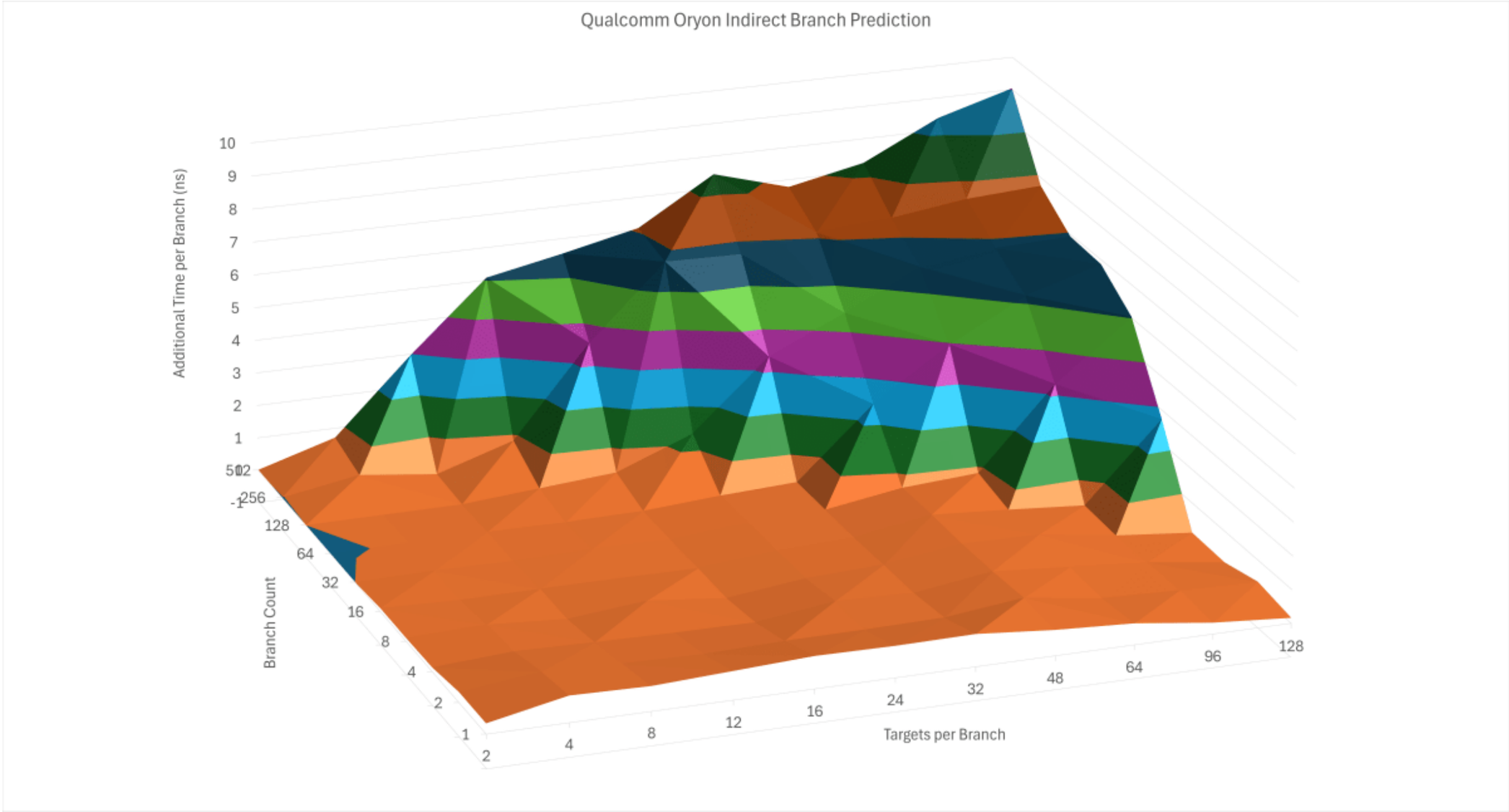
https://chipsandcheese.com/p/qualcomms-oryon-core-a-long-time-in-the-making

Branch Target Caching



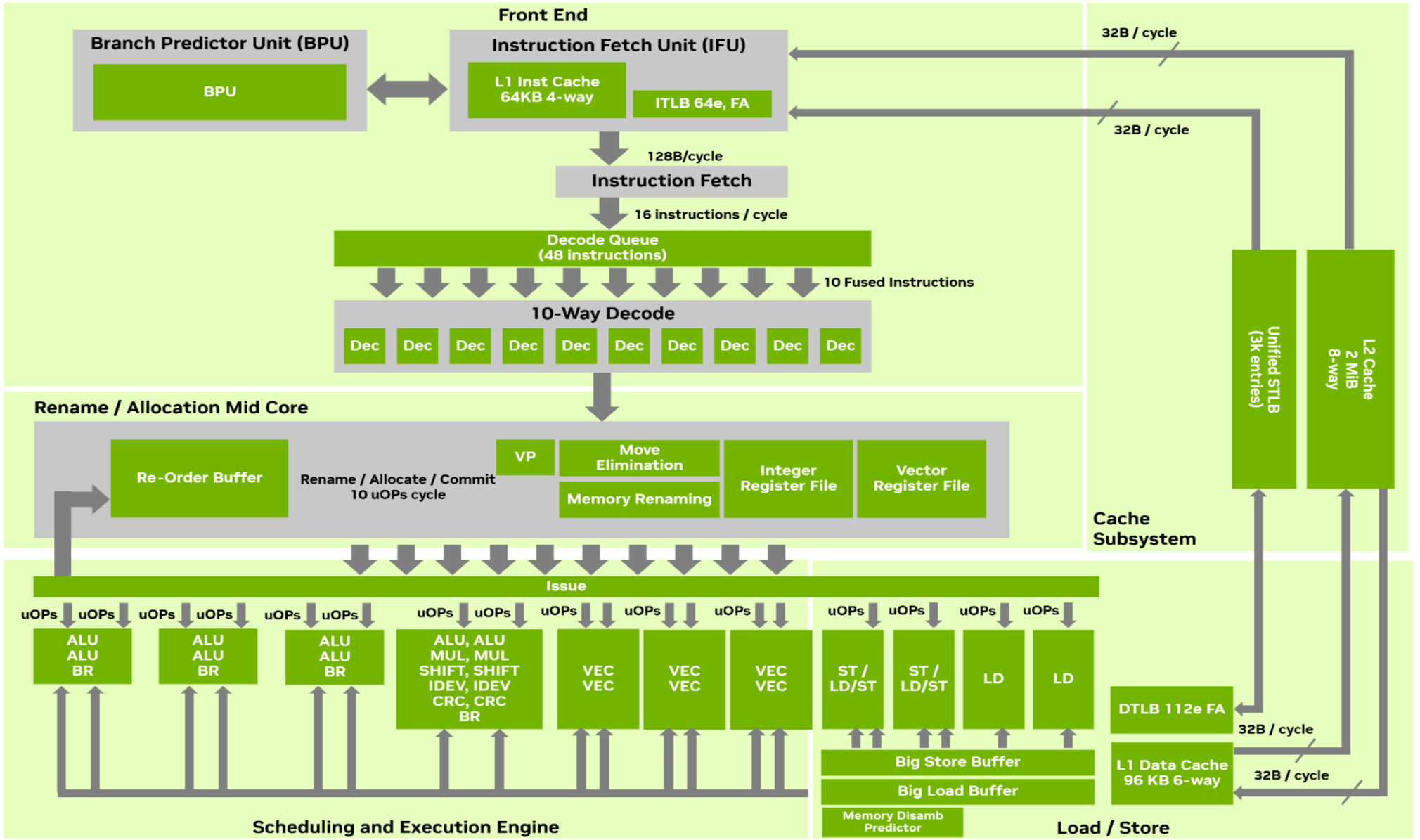
https://chipsandcheese.com/p/qualcomms-oryon-core-a-long-time-in-the-making

Indirect Branch Prediction



2025

NVIDIA Olympus Core Architecture



18-740: Modern Computer Architecture & Design

Lecture 5:
"Register Renaming & Out-of-Order Execution"

John Paul Shen
September 8, 2026

Next Time...

- Required Reading Assignment:
  - **Chapter 5 of Shen and Lipasti (SnL).**
- Recommended References:
  - Robert Tomasulo, "An Efficient Algorithm for Exploiting Multiple Arithmetic Units," IBM Journal of Research and Development, 11(1):25-33, January 1967.
  - Bo Zhang, "A Tomasulo's Algorithm Emulator," February 1, 2013.



Electrical & Computer ENGINEERING